INSTITUTE OF TECHNOLOGY AND MANAGEMENT SKILLS UNIVERSITY, KHARGHAR, NAVI MUMBAI

DSA 2: ASSIGNMENT 4

Prepared by:

Name of Student : Chaitanya Dalvi

Roll No: 19

Batch: 2023-27

Dept. of CSE

Q1. What do you mean by hashing and hash functions in Data Structures?

Hashing is a process of converting a given key into a unique value (index) that can be used to access the data faster. The data is stored in a structure called a hash table, and the process of converting the key to an index is done by a hash function.

A hash function maps the input key to an integer value, which is the index where the key-value pair will be stored in the hash table. The aim is to distribute keys evenly across the hash table to avoid collisions and improve lookup efficiency.

Q2. What are the types of collision resolution techniques in Data Structures?

Collision occurs when two keys hash to the same index. The primary collision resolution techniques are:

1. **Open Addressing:**
    - Linear Probing
    - Quadratic Probing
    - Double Hashing
2. **Chaining:**
    - Each slot in the hash table points to a linked list of keys that hash to the same index.

Q3. A hash function f defined as f(key) = key mod 7, with linear probing insert the keys 37, 38, 72, 48, 98, 11, 56. Into a table indexed from 0, in which location will the key 11 be stored?

Key 11 will be stored at index 5 after applying the linear probing technique as shown below:

- f(37) = 37 % 7 = 2 → Insert at index 2.
- f(38) = 38 % 7 = 3 → Insert at index 3.
- f(72) = 72 % 7 = 2 → Collision → Linear probing → Insert at index 4.
- f(48) = 48 % 7 = 6 → Insert at index 6.
- f(98) = 98 % 7 = 0 → Insert at index 0.
- f(11) = 11 % 7 = 4 → Collision → Linear probing → Insert at index 5.
- f(56) = 56 % 7 = 0 → Collision → Linear probing → Insert at index 1.

Q4. Explain the chaining method with an appropriate example.

Chaining is a collision resolution technique where each slot in the hash table points to a linked list (or chain) of elements that hash to the same index.

Example:

Consider a hash table of size 5 and keys: 10, 15, 20, 25, 30, using key mod 5 as the hash function:

- 10 % 5 = 0 → 10 is inserted at index 0.
- 15 % 5 = 0 → Collision → Add 15 to the linked list at index 0.
- 20 % 5 = 0 → Collision → Add 20 to the linked list at index 0.
- 25 % 5 = 0 → Collision → Add 25 to the linked list at index 0.
- 30 % 5 = 0 → Collision → Add 30 to the linked list at index 0.

At index 0, the linked list contains: 10 → 15 → 20 → 25 → 30.

Q5. What is the primary purpose of linear probing in hashing?

The primary purpose of **linear probing** is to resolve collisions in hash tables by searching sequentially for the next available slot in case of a collision.

Q6. How does linear probing handle collision?

In linear probing, when a collision occurs, the algorithm searches the hash table sequentially from the current index (where the collision happened) to find the next available empty slot.

Q7. Explain quadratic probing method of hashing.

In **quadratic probing**, when a collision occurs, instead of moving to the next sequential index (as in linear probing), the algorithm moves to the index calculated by adding successive powers of an integer i to the original hashed index.

This reduces primary clustering but may lead to secondary clustering.

Q8. What is the difference between linear probing and quadratic probing methods?

- **Linear Probing**: The next index is found by incrementing the current index by a constant value (usually 1). It tends to lead to primary clustering where large blocks of consecutive filled slots are formed.
- **Quadratic Probing**: The next index is found by adding successive powers of an integer (e.g., 1, 4, 9, etc.) to the current index. This reduces primary clustering but may still suffer from secondary clustering.

Q9. Apply Open Hashing Chaining method and insert the keys into the hash table.

Keys: 42, 19, 10, 12

Hash function: K mod 5

Table size: 5

Index	Chain (Linked List)
0	10
1	None
2	42, 12
3	None
4	19

Explanation:

- 42 % 5 = 2 → Insert 42 at index 2.
- 19 % 5 = 4 → Insert 19 at index 4.
- 10 % 5 = 0 → Insert 10 at index 0.
- 12 % 5 = 2 → Collision → Add 12 to the chain at index 2.

Q10. Insert keys at the first free location from (u + (i * i)) % m where i = 0 to (m - 1).

Keys: 3, 2, 9, 6, 11, 13, 7, 12

Hash function: h(k) = 2k + 3

Table size (m): 10

This problem uses **quadratic probing** with the division method. Here's how we insert the keys:

1. **Key 3**:
   h(3) = (2 * 3) + 3 = 9
   Insert at 9 % 10 = 9.

2. **Key 2**:
   h(2) = (2 * 2) + 3 = 7
   Insert at 7 % 10 = 7.

3. **Key 9**:
   $h(9) = (2 * 9) + 3 = 21$
   Insert at $21 \% 10 = 1$.

4. **Key 6**:
   $h(6) = (2 * 6) + 3 = 15$
   Insert at $15 \% 10 = 5$.

5. **Key 11**:
   $h(11) = (2 * 11) + 3 = 25$
   Insert at $25 \% 10 = 5$ (Collision)
   Quadratic probing:
   Try $(5 + 1^2) \% 10 = 6$ → Insert at index 6.

6. **Key 13**:
   $h(13) = (2 * 13) + 3 = 29$
   Insert at $29 \% 10 = 9$ (Collision)
   Quadratic probing:
   Try $(9 + 1^2) \% 10 = 0$ → Insert at index 0.

7. **Key 7**:
   $h(7) = (2 * 7) + 3 = 17$
   Insert at $17 \% 10 = 7$ (Collision)
   Quadratic probing:
   Try $(7 + 1^2) \% 10 = 8$ → Insert at index 8.

8. **Key 12**:
   $h(12) = (2 * 12) + 3 = 27$
   Insert at $27 \% 10 = 7$ (Collision)
   Quadratic probing:
   Try $(7 + 1^2) \% 10 = 8$ (Collision)
   Try $(7 + 2^2) \% 10 = 1$ (Collision)
   Try $(7 + 3^2) \% 10 = 6$ (Collision)
   Try $(7 + 4^2) \% 10 = 3$ → Insert at index 3.

Final Hash Table (using quadratic probing):

Index	Key

0	13
1	9
2	Empty
3	12
4	Empty
5	6
6	11
7	2
8	7
9	3